

Lab Checkpoint 1: stitching substrings into a byte stream

Due: Friday, October 8, 5 p.m.

Late deadline: Oct. 10, 11:59 p.m. (last day to receive feedback; 2/3 cap on style grade)

0 Collaboration Policy

The programming assignments must be your own work: You must write all the code you hand in for the programming assignments, except for the code that we give you as part of the assignment. Please do not copy-and-paste code from Stack Overflow, GitHub, or other sources. If you base your own code on examples you find on the Web or elsewhere, cite the URL in a comment in your submitted source code.

Working with others: You may not show your code to anyone else, look at anyone else's code, or look at solutions from previous years. You may discuss the assignments with other students, but do not copy anybody's code. If you discuss an assignment with another student, please name them in a comment in your submitted source code. Please refer to the course administrative handout for more details, and ask on Ed if anything is unclear.

Ed: Please feel free to ask questions on Ed, but please don't post any source code.

1 Overview

Suggestion: read the whole lab document before implementing.

In Lab 0, you used an *Internet stream socket* to fetch information from a website and send an email message, using Linux's built-in implementation of the Transmission Control Protocol (TCP). This TCP implementation managed to produce a pair of *reliable in-order byte streams* (one from you to the server, and one in the opposite direction), even though the underlying network only delivers "best-effort" datagrams. By this we mean: short packets of data that can be lost, reordered, altered, or duplicated. You also implemented the byte-stream abstraction yourself, in memory within one computer. Over the next four weeks, you'll implement TCP, to provide the byte-stream abstraction between a pair of computers separated by an unreliable datagram network.

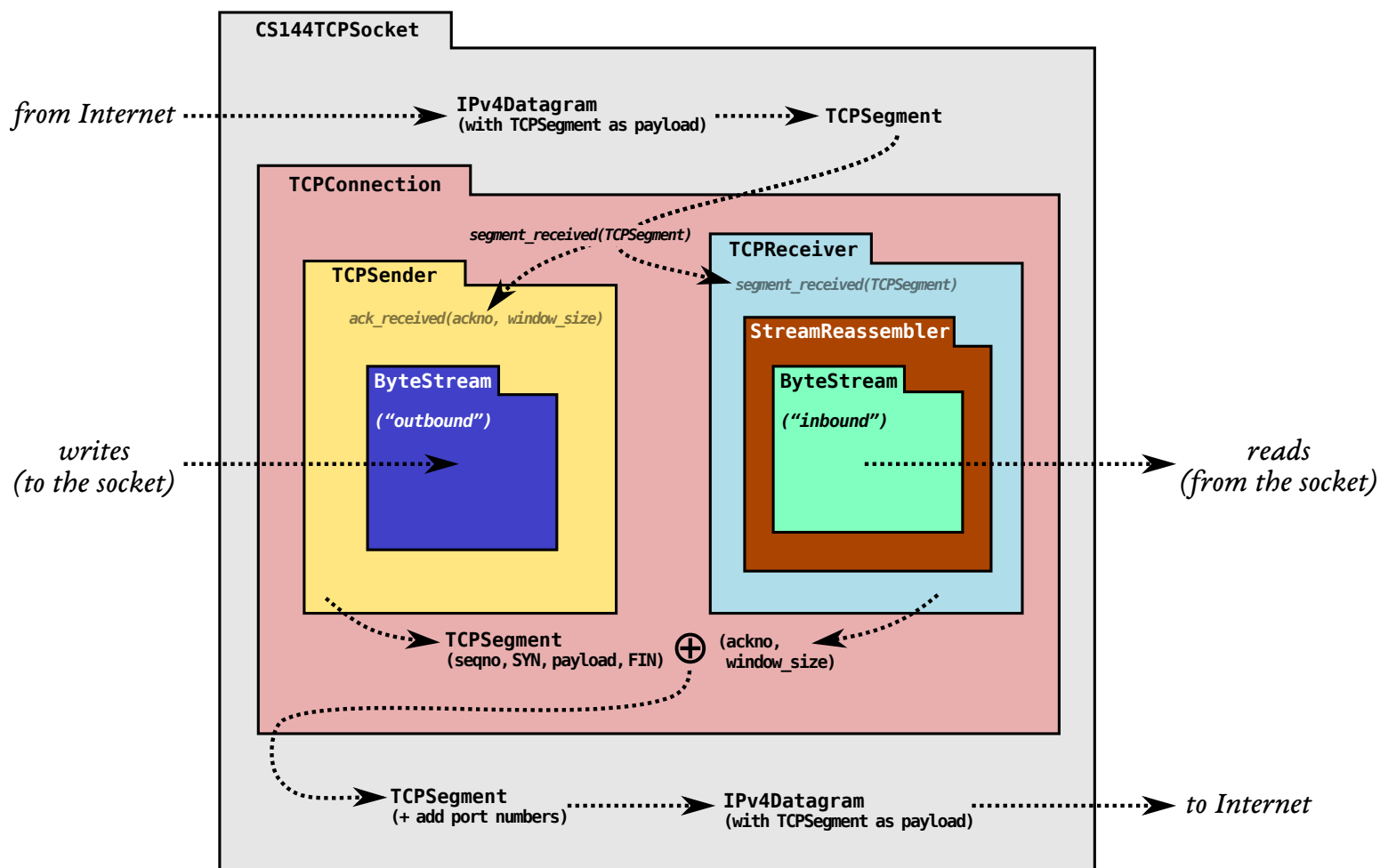


Figure 1: The arrangement of modules and dataflow in your TCP implementation. The `ByteStream` was Lab 0. The job of TCP is to convey two `ByteStreams` (one in each direction) over an unreliable datagram network, so that bytes written to the socket on one side of the connection emerge as bytes that can be read at the peer, and vice versa. Lab 1 is the `StreamReassembler`, and in Labs 2, 3, and 4 you'll implement the `TCPReceiver`, `TCPSender`, and then the `TCPConnection` to tie it all together.

★*Why am I doing this?* Providing a service or an abstraction on top of a different less-reliable service accounts for many of the interesting problems in networking. Over the last 40 years, researchers and practitioners have figured out how to convey all kinds of things—messaging and e-mail, hyperlinked documents, search engines, sound and video, virtual worlds, collaborative file sharing, digital currencies—over the Internet. TCP’s own role, providing a pair of reliable byte streams using unreliable datagrams, is one of the classic examples of this. A reasonable view has it that TCP implementations count as the most widely used nontrivial computer programs on the planet.

The lab assignments will ask you to build up a TCP implementation in a modular way. Remember the `ByteStream` you just implemented in Lab 0? In the next four labs, you’ll end up conveying two of them across the network: an “outbound” `ByteStream`, for data that a local application writes to a socket and that your TCP will send *to* the peer, and an “inbound” `ByteStream` for data coming *from* the peer that will be read by a local application. Figure 1 shows how the pieces fit together.

1. In Lab 1, you’ll implement a *stream reassembler*—a module that stitches small pieces of the byte stream (known as substrings, or **segments**) back into a contiguous stream of bytes in the correct sequence.
2. In Lab 2, you’ll implement the part of TCP that handles the inbound byte-stream: the `TCPReceiver`. This involves thinking about how TCP will represent each byte’s place in the stream—known as a “sequence number.” The `TCPReceiver` is responsible for telling the sender (a) how much of the inbound byte stream it’s been able to assemble successfully (this is called “acknowledgment”) and (b) how many more bytes the sender is allowed to send right now (“flow control”).
3. In Lab 3, you’ll implement the part of TCP that handles the outbound byte-stream: the `TCPSender`. How should the sender react when it suspects that a segment it transmitted was lost along the way and never made it to the receiver? When should it try again and re-transmit a lost segment?
4. In Lab 4, you’ll combine your work from the previous labs to create a working TCP implementation: a `TCPConnection` that contains a `TCPSender` and `TCPReceiver`. You’ll use this to talk to real servers around the world.

2 Getting started

Your implementation of TCP will use the same `Sponge` library that you used in Lab 0, with additional classes and tests. To get started:

1. Make sure you have committed all your solutions to Lab 0. Please don’t modify any files outside the top level of the `libsponge` directory, or `webget.cc`. You may have trouble merging the Lab 1 starter code otherwise.
2. While inside the repository for the lab assignments, run `git fetch` to retrieve the most recent version of the lab assignments.

3. Download the starter code for Lab 1 by running `git merge origin/lab1-startercode`.
4. Within your `build` directory, compile the source code: `make` (you can run, e.g., `make -j4` to use four processors when compiling).
5. Outside the `build` directory, open and start editing the `writups/lab1.md` file. This is the template for your lab writeup and will be included in your submission.

3 Putting substrings in sequence

In this and the next lab, you will implement a TCP receiver: the module that receives datagrams and turns them into a reliable byte stream to be read from the socket by the application—just as your `webget` program read the byte stream from the webserver in Lab 0.

The TCP sender is dividing its byte stream up into short *segments* (substrings no more than about 1,460 bytes apiece) so that they each fit inside a datagram. But the network might reorder these datagrams, or drop them, or deliver them more than once. The receiver must reassemble the segments into the contiguous stream of bytes that they started out as.

In this lab you'll write the data structure that will be responsible for this reassembly: a `StreamReassembler`. It will receive substrings, consisting of a string of bytes, and the index of the first byte of that string within the larger stream. **Each byte of the stream** has its own unique index, starting from zero and counting upwards. The `StreamReassembler` will own a `ByteStream` for the output: as soon as the reassembler knows the next byte of the stream, it will write it into the `ByteStream`. The owner can access and read from the `ByteStream` whenever it wants.

Here's what the interface looks like:

```
// Construct a `StreamReassembler` that will store up to `capacity` bytes.
StreamReassembler(const size_t capacity);

// Receive a substring and write any newly contiguous bytes into the stream,
// while staying within the memory limits of the `capacity`. Bytes that would
// exceed the capacity are silently discarded.
//
// `data`: the substring
// `index` indicates the index (place in sequence) of the first byte in `data`
// `eof`: the last byte of this substring will be the last byte in the entire stream
void push_substring(const string &data, const uint64_t index, const bool eof);

// Access the reassembled ByteStream (your code from Lab 0)
ByteStream &stream_out();

// The number of bytes in the substrings stored but not yet reassembled
size_t unassembled_bytes() const;
```

```
// Is the internal state empty (other than the output stream)?  
bool empty() const;
```

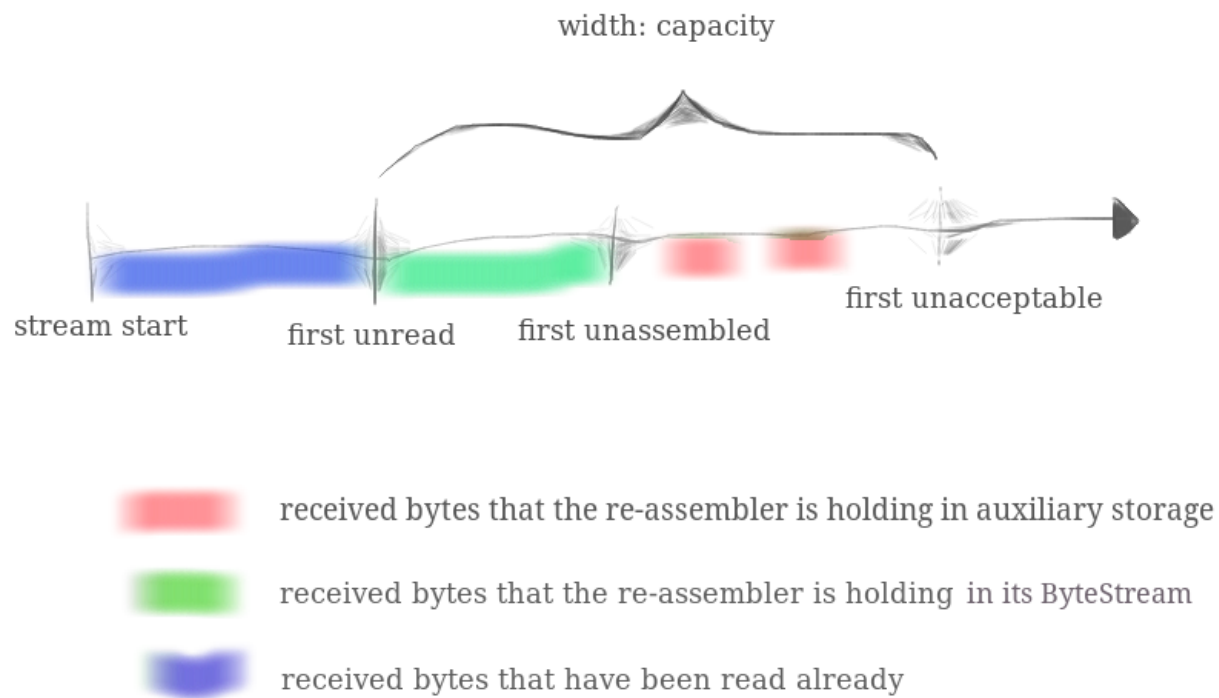
★*Why am I doing this?* TCP robustness against reordering and duplication comes from its ability to stitch arbitrary excerpts of the byte stream back into the original stream. Implementing this in a discrete testable module will make handling incoming segments *much* easier.

The full (public) interface of the reassembler is described by the `StreamReassembler` class in the `stream_reassembler.hh` header. Your task is to implement this class. You may add any private members and member functions you desire to the `StreamReassembler` class, but you cannot change its public interface.

3.1 What's the “capacity”?

Your `push_substring` method will ignore any portion of the string that would cause the `StreamReassembler` to exceed its “capacity”: a limit on memory usage, i.e. the maximum number of bytes it is ever allowed to store. This prevents the reassembler from using an unbounded amount of memory, no matter what the TCP sender decides to do. We've illustrated this in the picture below. The “capacity” is an upper bound on *both*:

1. The number of bytes in the reassembled `ByteStream` (shown in green below), and
2. The maximum number of bytes that can be used by “unassembled” substrings (shown in red)



You may find this picture useful as you implement the `StreamReassembler` and work through the tests—it's not always natural what the "right" behavior is.

3.2 FAQs

- *What is the index of the first byte in the whole stream?* Zero.
- *How efficient should my implementation be?* Please don't take this as a challenge to build a grossly space- or time-inefficient data structure—this data structure will be the foundation of your TCP implementation. A ballpark expectation would be that each of the new Lab 1 tests can complete in less than half a second.
- *How should inconsistent substrings be handled?* You may assume that they don't exist. That is, you can assume that there is a unique underlying byte-stream, and all substrings are (accurate) slices of it.
- *What may I use?* You may use any part of the standard library you find helpful. In particular, we expect you to use at least one data structure.
- *When should bytes be written to the stream?* As soon as possible. The only situation in which a byte should not be in the stream is that when there is a byte before it that has

not been “pushed” yet.

- *May substrings provided to the `push_substring()` function overlap?* Yes.
- *Will I need to add private members to the `StreamReassembler`?* Yes. Substrings may arrive in any order, so your data structure will have to “remember” substrings until they’re ready to be put into the stream—that is, until all indices before them have been written.
- *Is it okay for our re-assembly data structure to store overlapping substrings?* No. It is possible to implement an “interface-correct” reassembler that stores overlapping substrings. But allowing the re-assembler to do this undermines the notion of “capacity” as a memory limit. We’ll consider the storage of overlapping substrings to be a style violation when grading.
- *More FAQs:* For more, please see https://cs144.github.io/lab_faq.html.

4 Development and debugging advice

1. You can test your code (after compiling it) with `make check_lab1`.
2. Please re-read the section on “using Git” in the Lab 0 document, and remember to keep the code in the Git repository it was distributed in on the `master` branch. Make small commits, using good commit messages that identify what changed and why.
3. Please work to make your code readable to the CA who will be grading it for style. Use reasonable and clear naming conventions for variables. Use comments to explain complex or subtle pieces of code. Use “defensive programming”—explicitly check preconditions of functions or invariants, and throw an exception if anything is ever wrong. Use modularity in your design—identify common abstractions and behaviors and factor them out when possible. Blocks of repeated code and enormous functions will make it hard to follow your code.
4. Please also keep to the “Modern C++” style described in the Lab 0 document. The `cppreference` website (<https://en.cppreference.com>) is a great resource, although you won’t need any sophisticated features of C++ to do these labs. (You may sometimes need to use the `move()` function to pass an object that can’t be copied.)
5. If you get a segmentation fault, something is really wrong! We would like you to be writing in a style where you use safe programming practices to make segfaults extremely unusual (no `malloc()`, no `new`, no pointers, safety checks that throw exceptions where you are uncertain, etc.). That said, to debug you can configure your build directory with `cmake .. -DCMAKE_BUILD_TYPE=RelASan` to enable the compiler’s “sanitizers” to detect memory errors and undefined behavior and give you a nice diagnostic about when they occur. You can also use the `valgrind` tool. You can also configure with `cmake .. -DCMAKE_BUILD_TYPE=Debug` and use the GNU debugger (`gdb`). But please

remember that these options (especially the sanitizers) will slow down compilation and execution—you don't want to accidentally leave them in place!

6. You can reset the build system with `make clean` and `cmake .. -DCMAKE_BUILD_TYPE=Release`. Or if you get your builds really stuck and aren't sure how to fix them, you can erase your build directory (`rm -rf build`—please be careful not to make a typo as this will erase whatever you tell it), make a new build directory, and `cmake ..` again.

5 Submit

1. In your submission, please only make changes to the `.hh` and `.cc` files in the top level of `libsponge`. Within these files, please feel free to add private members as necessary, but please don't change the *public* interface of any of the classes.
2. Before handing in any assignment, please run these in order:
 - (a) `make format` (to normalize the coding style)
 - (b) `make` (to make sure the code compiles)
 - (c) `make check_lab1` (to make sure the automated tests pass)
3. Write a report in `writeups/lab1.md`. This file should be a roughly 20-to-50-line document with no more than 80 characters per line to make it easier to read. The report should contain the following sections:
 - (a) **Program Structure and Design.** Describe the high-level structure and design choices embodied in your code. You do not need to discuss in detail what you inherited from the starter code. Use this as an opportunity to highlight important design aspects and provide greater detail on those areas for your grading TA to understand. You are strongly encouraged to make this writeup as readable as possible by using subheadings and outlines. Please do not simply translate your program into an paragraph of English.
 - (b) **Implementation Challenges.** Describe the parts of code that you found most troublesome and explain why. Reflect on how you overcame those challenges and what helped you finally understand the concept that was giving you trouble. How did you attempt to ensure that your code maintained your assumptions, invariants, and preconditions, and in what ways did you find this easy or difficult? How did you debug and test your code?
 - (c) **Remaining Bugs.** Point out and explain as best you can any bugs (or unhandled edge cases) that remain in the code.
4. In your writeup, please also fill in the number of hours the assignment took you and any other comments.

5. When ready to submit, please follow the instructions at <https://cs144.github.io/submit>. Please make sure you have committed everything you intend before submitting.
6. Please let the course staff know ASAP of any problems at the Tuesday-evening lab session, or by posting a question on Ed. Good luck!